

UNIVERSIDAD DE LOS ANDES

FACULTAD DE INGENIERÍA

Ingeniería de Sistemas y Computación

Proyecto de Grado 2026-20

QuickR

Mateo López Cuartas

November 13, 2026

Bogotá D.C., Colombia

Contents

1 Introduction **2**
- 1.1 Definitions . . . 2
- 1.2 Context . . . 2
- 1.3 Motivation . . . 4
- 1.4 Objectives . . . 5
  - 1.4.1 Main Objective . . . 5
  - 1.4.2 Secondary Objectives . . . 5

2 Methodology **5**
- 2.1 State of the Art . . . 5
- 2.2 QuickR . . . 9
- 2.3 Implementation . . . 9

3 Results and Discussion **9**
- 3.1 Results . . . 9
- 3.2 Conclusion . . . 9

1 Introduction

1.1 Definitions

- **Streaming:** The process of transmitting and/or receiving media content over the internet in real time and in a continuous manner.
- **Open Source Program:** Software and its source code for which it is publicly available to view, use, distribute, and modify without restriction.
- **Closed Source Program:** Software whose source code is either partially or fully restricted from public access, which is usually developed and maintained by a private company or organization.

1.2 Context

Ever since the turn of the millenium, the internet has had a significant impact on global communication technologies. By the year 2000, 51% of global telecommunications networks were transmitted via the internet, a figure that rose to 97% by 2007 [1]. This reliance on the internet has been steadily increasing over time, culminating in 2020 when quarantine and lockdown measures due to the pandemic made the internet an essential part of daily life for most people worldwide. This drastic change in people's lives led to an all-time high demand for applications that connect different individuals through digital means, such as Meta's Metaverse, the increased demand for virtual meeting software (Zoom, Google Meet, Microsoft Teams), and mass communication services like Discord and Slack.

It is in this context that the need to adapt these daily activities to a new digital environment arises; in this case, broadcasting and sharing multimedia content via streaming from one device (host) for others to view on their devices in real time, regardless of their geographical location. Media streaming became popular through formal or academic means such as Zoom and Microsoft Teams, while the more casual media streaming needs were dominated by platforms like Twitch and Discord (the latter, although primarily used as a social messaging platform, also offers VoIP and video streaming services, which is the relevant service for this paper). The rapid expansion of video traffic on the internet, coupled with the staggering amount of users of these services, has created a strong need to develop robust protocols that work

quickly and efficiently under any type of connectivity conditions, as well as infrastructures that are sufficiently robust and scalable to meet the demand for these transmissions.

This has led to an interest by developers to restrict in certain ways some functionalities and features offered by these streaming services in a way that is actually profitable. Paywalls and subscription models are some of the most common ways companies and services implement a restriction that solves multiple problems at the same time: first of all, it limits the number of users that can use the service at any given moment, which reduces maintenance costs and bandwidth usage; second, it produces a source of income for the service provider, which can be used to improve the service and infrastructure; and third, it helps create a sense of exclusivity to the paying user. Some services that include these kinds of subscriptions are Discord, Google Meet, and Zoom. In the case of Discord, the service offers a subscription called Nitro that, besides other benefits, allows the user to upgrade the stream quality from the free tier cap of 720p x 30fps to a maximum of 4k x 60fps, with a monthly cost of $9.99 USD (as of November 2026). With Google Meet, the free tier besides allowing a call for only up to 60 minutes, hard limits the stream quality to 720p x 30fps, while any of Google's subscriptions allows for a maximum of 1080p x 30fps. Zoom, on the other hand, does not really restrict the stream quality, allowing content to be shared in the native resolution at 30fps for no extra cost, but it does recommend downscaling to 1080p to save on bandwith. It also limits the free tier to 40 minutes per call, restricting the user not by the quality of the stream, but by the time they can use it.

Given the context of these service models with their constraints, it becomes apparent that users, even though there are multiple options for meeting their streaming needs, are still limited by the restrictions imposed by these companies. Open source projects are alternatives to these services, but even though they can be more beneficial than closed-source by improving security, allowing way more flexibility, and fostering community involvement by having collaborative development [2], there are many chases in which these alternativesare not as widespread and used as the commercial ones. This lower popularity directly reduces the appeal to the common user to move onto these alternatives. Regardless, there exist a myriad of open source projects that act as cornerstones of the modern internet and computing as a whole. Some of the main examples typically include the Linux kernel, one of, if not the most, important operating system kernels worldwide, the Apache

web server, MySQL, and PHP, all four being the main components of the LAMP stack. And even outside of the scope of giant corporations running open source, even more projects exist that are maintained by small teams or even individuals. There are many cases of these projects existing without an explicit commercial or financial interest, but rather as a means to solve a specific need, desire or problem someone encountered. Hausberg et al. [3] states how motivations behind OSS (Open Source Software) contributions stem from the individual's desire to contribute towards innovation that becomes a common good. It can be seen as the rational choice to provide a contribution for free, as the result afterwards becomes a private, ongoing benefit. It is through this reasoning that one may explain the reasoning behind these projects taking off and staying relevant, even in the face of competition from commercial alternatives.

1.3 Motivation

Due to the large amount of corporate-maintained software on many aspects of every day technological life, there has always been an interest on open source software development and maintenance. The democratization of software presents a unique opportunity for developers to create tailor-made solutions to their own problems or needs, benefitting from having a community of developers and users that can contribute to the project either by providing feedback, reporting bugs, contributing code, or by financially backing the project. Besides from the characteristic of it being a private or open source project, developing a streaming service application involves a considerable amount of technical knowledge and integration in order to provide a robust and efficient alternative for the user. Since it involves concepts of networking, video and audio encoding, synchronization, and UI/UX design, working on a streaming service application is a great way to learn by creating something tangible and useful, while also allowing a community of users to both support and benefit from it. Another motivation stems from more personal reasons, as the distaste of corporate software coupled with the desire to watch movies with people who are geographically far away made way for this project idea to be born.

1.4 Objectives

For this project, there will be a single main objective and a set of secondary objectives:

1.4.1 Main Objective

For this projetc, the main objetive is to design and develop a desktop application that implements a video streaming protocol that allows users to share real-time video and audio between devices from scratch.

1.4.2 Secondary Objectives

The secondary or auxiliary objectives are the following:

- Develop a Peer-to-Peer (P2P) architecture that allows users to communicate between each other's devices.
- Implement a full transmission pipeline over UDP that allows for a reliable packet transmission.
- Build a user interface that shows received video and audio streams in real time.
- Compare the performance between state-of-the-art streaming tools and the developed application.

2 Methodology

For the development of this project, the methodology will start with a review of the state-of-the-art streaming tools and protocols, exploring the solutions that are already on the market. After that, the next step will be the design and eventual implementation of the application, to be followed by a series of tests and performance comparisons agains the existing alternatives using packet analysis tools such as Wireshark.

2.1 State of the Art

- **Parsec:** Parsec is a desktop application whose main goal is to allow for a seamless experience using remote desktop connections, with online

gaming as the primary focus, but allowing for connections to cloud devices and personal computers as well. According to the information provided by the official Parsec Team (including 2 different blog posts and a talk at Code Driven NYC), this application's streaming logic was built based these priorities in the following order of importance: **latency, then frame rate, then video quality**[4]. To achieve the goal set on reducing latency to the absolute minimum, many standard technologies were tested, starting with both UDP and TCP protocols. The developers took both protocols and decided which attributes were desirable to bring over to a new, proprietary protocol that met all of the needs necessary for Parsec to provide a seamless video game streaming experience:

TCP:

1. Reliability and reordering.
2. Stream Semantics
3. Connection state and handshake via DTLS
4. ACK to confirm packet reception

UDP:

1. Stateless hole-punching

Due to the necessity of bringing some (but not all) characteristics of each protocol, the Parsec team developed their own protocol called BUD (Better User Datagrams) for their service. This protocol works by diverging from the standard TCP behavior of working in any network condition by throttling the performance to give priority to the correct delivery of the packets. Instead, it assumes the user has more bandwidth than a tested minimum (not a concrete value, but according to the Code Driven NYC talk, it is $\sim$ 30 Mbps with less than 50 ping). By using this assumption, the algorithm does not have to concern itself with cases of poor connection but rather works with known stable connections.

The most important characteristics regarding the BUD protocol are the following:

- NAT traversal by hole punching: To bypass firewalls blocking incoming requests by the incoming device, the protocol uses a staple functionality from UDP to establish direct Peer to Peer connection.
- Congestion control not done via internal packet sending limits, but instead via the encoder. Using H.264 as the main video encoder, Parsec controls its bitrate frequently depending on the network congestion and RTT (Round-Trip Time). By varying the bitrate, the algorithm can reduce load on the bandwidth without halting packet sending, so that the client can still receive data and show live updates on the screen without any added latency.
- Fast packet retransmission: By having ACK packets and a buffer that loads packets and updates them at runtime (which is another benefit of never having to reallocate memory), the algorithm can notice quickly if a packet N hasn't received an ACK while packet $N+1$ already received its own ACK. When this happens, it triggers a quick retransmission of packet N to avoid any delays. There is also a slow retransmition as a failsafe.
- A very concise and small packet header with Sequence Number, Data Type and Payload.

With these characteristics, BUD is able to perform with an exceptional quality in various environments. During live showcases, Parsec was able to perform at near 30 fps with packet loss between 5 and 10%. Nonetheless, the documents state the software still falls behind on symmetric NATs (they restrict the hole punching method needed for the first connection establishment), degraded networks are still not contemplated for the algorithm's performance, and creating a protocol with custom tools brings a great deal of technical debt and issues that have to be solved in-house.

- **Discord:** Although the Discord Application offers a myriad of services, mainly focused on instant messaging and VoIP communications, it also includes a feature called *Go Live*, which is the company's feature for screen sharing inside of the app's voice channel. According to the blog detailing this service's inner workings, *Go Live* works with a pipeline that relies on three different but interconnected components: **Screen capture, data encoding, and transport.** [5]

1. **Capture:** For capture to go as smooth as possible, the algorithm checks available capture methods the device's operating system provides, and picks the best one available at the moment. If that capture method fails, the system falls back into the next best alternative to keep the stream going without any issues. Some of the mentioned methods include the OS's own APIs, or injecting a DLL into the game to read what the screen renders and sending it directly to the pipeline. For the audio component, the app captures the audio of the window that was selected for sharing, but no audio is sent if the entire device's screen gets selected. No definite reasoning is provided for this, but a reasonable explanation would be that, since the main goal of screen sharing is to be used inside of VoIP calls, sharing the entire device's audio would probably mirror the other participants' voice audio, worsening the experience for the clients that are watching the screen stream.
2. **Encoding:** Since a single 1080p frame can be as large as 6MB, an encoder is completely necessary to avoid bottlenecking the network socket and to allow frames to be sent at a faster rate. Every time a *Go Live* session is iniciated, all clients negotiate a codec that can be used to decode the packets by every participant of the call. The algorithm prioritizes hardware encoders (from GPUs) over software encoders whenever possible to attempt to reduce CPU load. The more commonly available codecs are VP8 and H.264, with HEVC and AV1 being available on some devices. When the network quality degrades, the algorithm drops frames to relieve congestion until it improves.
3. **Data Transmission:** Unlike Parsec's system, *Go Live* is not Peer-to-Peer. Instead, it relies on Discord's own servers and CDNs to get each participant's IP address and relaying the data packets to the correct recipients at all points of the process. This is primarily done due to privacy purposes, as users do not get access to others' private information even if they are on the same call. This transport is built over WebRTC with video and audio packets being sent separately. The sending rate is dynamically capped by the user with the slowest network, so everyone gets the same experience.

When the receiver gets the video and audio packets, it is them who

synchronize both of these streams on the client device before showing it to the user. A known issue based on the technical decisions taken for this approach is that the VoIP audio and the stream are not synchronized, as they are managed by different systems. This puts pressure on the developers to reduce stream latency as much as possible to avoid degrading the users' experience with the service.

2.2 QuickR

2.3 Implementation

3 Results and Discussion

3.1 Results

3.2 Conclusion

References

[1] M. Hilbert and P. López, "The world's technological capacity to store, communicate, and compute information," *Science*, vol. 332, no. 6025, pp. 60–65, 2011, doi: 10.1126/science.1200970. eprint: https://www.science.org/doi/pdf/10.1126/science.1200970. [Online]. Available: https://www.science.org/doi/abs/10.1126/science.1200970

[2] J. Madge and A. Bennett. "Open-source software: Why it matters and how to get involved," The Alan Turing Institute Blog. [Online]. Available: https://www.turing.ac.uk/blog/open-source-software-why-it-matters-and-how-get-involved

[3] J. P. Hausberg and S. Spaeth, "Why makers make what they make: Motivations to contribute to open source hardware development," *R & D management*, vol. 50, no. 1, pp. 75–77, 2020.

[4] "A networking protocol built for the lowest latency interactive game streaming," Parsec Team, Accessed: Nov. 12, 2026. [Online]. Available: https://parsec.app/blog/a-networking-protocol-built-for-the-lowest-latency-interactive-game-streaming-1fd5a03a6007

[5] S. J. "How it all goes live: An overview of discord's streaming technology," Discord Engineering & Developers Blog. [Online]. Available: https://discord.com/blog/how-it-all-goes-live-an-overview-of-discords-streaming-technology